

EmotionBudgetApp 프로젝트 구조 및 핵심 코드 설명

Android 프로그래밍 기말 프로젝트
김동민 / 2271180

1. 프로젝트 개요

가계부를 쓰다 보면 어디에 돈을 썼는지는 알 수 있지만, 그때 왜 결제했는지는 기억하기 어려운 경우가 많다. EmotionBudgetApp은 이 점에서 출발한 앱이다. 수입과 지출을 기록하는 기본 가계부에 소비 당시의 감정을 함께 남겨, 지출 금액뿐 아니라 반복되는 소비 습관도 확인할 수 있도록 만들었다.

사용자는 기록 화면에서 금액, 날짜, 카테고리, 감정과 메모를 입력한다. 입력한 내용은 통계 화면에서 날짜별·주별·월별로 다시 볼 수 있고, 감정 화면에서는 감정별 지출 금액과 횟수를 비교할 수 있다. 예를 들어 스트레스를 받을 때 카페 지출이 반복되었다면 해당 패턴과 지난달보다 늘어난 횟수를 확인할 수 있다.

앱의 화면은 기록, 통계, 감정 세 부분으로 나누었다. 자주 사용하는 입력과 검색은 기록 화면에 모으고, 자세한 분석은 별도 화면으로 빼서 기록이 많아져도 필요한 내용을 찾기 쉽게 했다.

2. 개발 환경 및 사용 라이브러리

사용 항목	사용 내용
Kotlin 2.2.10	기록 추가, 검색 조건 처리, 통계 계산 등 앱의 동작을 작성하는 데 사용했다.
Android Gradle Plugin 9.1.1	앱 모듈의 SDK 버전과 빌드 설정을 관리한다.
Jetpack Compose	XML 레이아웃 대신 Kotlin 코드로 화면을 구성했다.
Material3	입력창, 버튼, 카드, 날짜 선택 창과 하단 네비게이션을 구성했다.
Activity Compose	<code>MainActivity</code> 에서 Compose 화면을 시작하기 위해 사용했다.
Lifecycle ViewModel Compose	화면이 바뀌어도 같은 기록 목록을 사용할 수 있도록 ViewModel을 연결했다.
StateFlow	기록이 추가·수정·삭제되면 목록과 통계 화면이 함께 갱신되도록 사용했다.

3. 프로젝트 구조

```
EmotionBudgetApp/
├── app/
│   ├── build.gradle.kts
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── res/
│       └── java/com/example/emotionbudgetapp/
│           ├── MainActivity.kt
│           ├── data/
│           │   └── Expense.kt
│           ├── viewmodel/
│           │   └── ExpenseViewModel.kt
│           └── ui/
│               ├── EmotionBudgetAppRoot.kt
│               ├── ExpenseScreen.kt
│               ├── ExpenseItem.kt
│               ├── LedgerReportScreen.kt
│               └── EmotionAnalysisScreen.kt
└── gradle/
    └── libs.versions.toml
```

파일	역할
<code>app/build.gradle.kts</code>	SDK 버전, Compose 사용 여부, 라이브러리 의존성을 설정한다.
<code>gradle/libs.versions.toml</code>	프로젝트에서 사용하는 라이브러리와 플러그인 버전을 관리한다.

파일	역할
MainActivity.kt	앱이 실행될 때 처음 호출되는 Activity이다.
data/Expense.kt	수입·지출 기록 한 건의 데이터 형식을 정의한다.
viewModel/ExpenseViewModel.kt	기록 목록과 추가, 수정, 삭제, 합계 계산을 담당한다.
ui/EmotionBudgetAppRoot.kt	하단 탭을 이용해 기록, 통계, 감정 화면을 전환한다.
ui/ExpenseScreen.kt	기록 입력과 검색 조건을 처리하고 기록 목록을 표시한다.
ui/ExpenseItem.kt	기록 목록의 개별 항목을 카드 형태로 표시한다.
ui/LedgerReportScreen.kt	일별, 달력, 주별, 월별 통계 화면을 표시한다.
ui/EmotionAnalysisScreen.kt	감정별 지출 금액과 횟수를 계산하고 그래프로 표시한다.

3.1 데이터와 화면의 연결

앱에서 기록 한 건은 `Expense` 객체로 저장된다. 기록 화면에서 새 내용을 입력하면 `ExpenseViewModel`이 객체를 만들어 목록에 추가한다. `ViewModel`이 가지고 있는 목록은 `StateFlow`로 공개되어 있으며, 기록·통계·감정 화면이 같은 목록을 읽는다. 이 때문에 기록 화면에서 값을 수정하거나 삭제하면 별도의 새로고침 과정 없이 다른 화면의 합계와 그래프도 바뀐다.

화면 코드는 역할에 따라 나누었다. `ExpenseScreen`은 입력, 검색, 기록 목록처럼 사용자가 직접 조작하는 기능을 맡는다. `LedgerReportScreen`은 날짜를 기준으로 기록을 묶어 보여주며, `EmotionAnalysisScreen`은 지출 기록에서 감정 정보를 골라 통계를 계산한다. 공통 데이터는 `ViewModel`에 두고, 각 화면에서는 필요한 방식으로 가공해 사용하도록 구성했다.

4. 주요 기능

기능	설명
수입·지출 기록	금액, 유형, 날짜, 카테고리, 감정, 메모를 입력한다.
기록 수정·삭제	기존 기록을 다시 입력창에 불러오거나 삭제할 수 있다.
기록 검색	검색어, 기간, 유형, 카테고리, 감정, 금액 범위로 기록을 찾는다.
통계 화면	기록을 일별, 달력, 주별, 월별, 요약 형태로 정리한다.
감정 분석	감정별 총지출, 소비 횟수, 평균 금액과 전월 대비 횟수 변화를 보여준다.
감정 소비 진단	스트레스 소비의 횟수와 비중을 계산하여 진단 문구를 표시한다.

4.1 기록 화면

기록 화면 상단에서는 현재까지 입력된 수입, 지출, 잔액과 기록 수를 한 번에 볼 수 있다. 새 기록 영역에서 수입 또는 지출을 선택한 뒤 금액과 날짜를 입력한다. 지출일 때는 카테고리나 감정을 함께 고르며, 필요하면 메모를 남길 수 있다. 수입은 감정 분석 대상이 아니므로 감정 선택 대신 수입 카테고리에 집중하도록 입력 항목을 구분했다.

저장된 기록은 카드 목록으로 표시된다. 수입은 파란색 + , 지출은 빨간색 - 으로 보여서 목록을 빠르게 구분할 수 있다. 수정 버튼을 누르면 기존 내용이 입력 영역으로 다시 들어가며, 삭제 버튼을 누르면 확인 창을 거친 뒤 기록이 삭제된다.

기록 찾기 영역에서는 메모와 금액, 날짜, 유형, 카테고리, 감정을 검색할 수 있다. 검색어 외에도 기간과 최소·최대 금액을 지정할 수 있으며, 결과가 있으면 조건에 맞는 기록 카드를 바로 아래에 표시한다. 결과가 없을 때는 빈 목록만 보여주는 대신 조건에 맞는 기록이 없다는 문장을 표시한다.

4.2 통계 화면

통계 화면은 선택한 달의 수입, 지출, 잔액을 먼저 보여준다. 좌우 이동 버튼으로 월을 바꿀 수 있으며 일별, 달력, 주별, 월별, 요약 탭으로 같은 기록을 여러 방식으로 확인할 수 있다.

일별 탭에서는 같은 날짜의 기록을 한 그룹으로 묶고 그날의 수입과 지출 합계를 함께 표시한다. 달력 탭은 날짜별 합계를 한 달 달력 형태로 보여주며, 주별 탭에서는 몇 주차에 지출이 많았는지 비교할 수 있다. 월별 탭과 요약 탭에서는 카테고리별 금액, 자주 나온 감정과 같은 내용을 정리한다.

4.3 감정 분석 화면

감정 화면은 지출 기록만 사용해 계산한다. 감정별 총지출, 소비 횟수, 평균 금액, 전체 지출에서 차지하는 비율을 보여주며, 금액과 횟수는 막대그래프로도 비교할 수 있다. 같은 감정의 지난달 기록 수를 함께 계산해 이번 달에 소비 횟수가 늘었는지도 표시한다.

상단의 감정 소비 진단 카드는 스트레스 소비 횟수, 전월 대비 증가 여부, 지출 비중과 대표 감정을 기준으로 위험도를 계산한다. 단순히 금액만 보여주는 것보다 사용자가 어떤 소비를 다시 살펴봐야 하는지 알 수 있도록 진단 문장과 추천 행동도 함께 표시한다.

5. 핵심 코드 설명

5.1 앱 시작과 Compose 화면 호출

파일: app/src/main/java/com/example/emotionbudgetapp/MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            val expenseViewModel: ExpenseViewModel = viewModel()
            EmotionBudgetAppRoot(viewModel = expenseViewModel)
        }
    }
}
```

MainActivity는 앱의 시작점이다. onCreate()에서 setContent를 호출하여 Compose 화면을 표시한다.

viewModel()로 생성한 ExpenseViewModel은 기록 목록을 관리하며, 루트 화면에 전달되어 각 화면에서 함께 사용된다.

5.2 수입·지출 데이터 클래스

파일: app/src/main/java/com/example/emotionbudgetapp/data/Expense.kt

```
enum class TransactionType(val label: String) {
    INCOME("수입"),
    EXPENSE("지출")
}

data class Expense(
    val id: Int,
    val amount: Int,
    val category: String,
    val emotion: String,
    val memo: String,
    val dateMillis: Long,
    val type: TransactionType = TransactionType.EXPENSE
)
```

TransactionType은 기록이 수입인지 지출인지 구분한다. Expense는 기록 한 건에 필요한 값을 모아 둔 데이터 클래스이다. 목록에서 특정 기록을 수정하거나 삭제할 때는 id를 사용하고, 날짜별·감정별 통계를 계산할 때는 dateMillis와 emotion 값을 사용한다.

5.3 ViewModel의 기록 목록 관리

파일: app/src/main/java/com/example/emotionbudgetapp/viewmodel/ExpenseViewModel.kt

```
private val _expenses = MutableStateFlow<List<Expense>>>(emptyList())
val expenses: StateFlow<List<Expense>>> = _expenses
private var nextId = 1

fun addExpense(amount: Int, category: String, emotion: String,
```

```

        memo: String, dateMillis: Long, type: TransactionType) {
    val newExpense = Expense(
        id = nextId++,
        amount = amount,
        category = category,
        emotion = emotion,
        memo = memo,
        dateMillis = dateMillis,
        type = type
    )
    _expenses.value = _expenses.value + newExpense
}

fun updateExpense(id: Int, amount: Int, category: String, emotion: String,
    memo: String, dateMillis: Long, type: TransactionType) {
    _expenses.value = _expenses.value.map { expense ->
        if (expense.id == id) {
            expense.copy(amount = amount, category = category, emotion = emotion,
                memo = memo, dateMillis = dateMillis, type = type)
        } else {
            expense
        }
    }
}

fun deleteExpense(expense: Expense) {
    _expenses.value = _expenses.value.filter { it.id != expense.id }
}

```

기록 목록은 `MutableStateFlow<List<Expense>>`로 관리한다. 기록 추가는 기존 목록 뒤에 새 기록을 붙이고, 수정은 `map`과 `copy`를 이용해 같은 `id`의 기록만 바꾼다. 삭제는 `filter`를 이용해 선택한 `id`를 제외한 기록만 남긴다.

5.4 입력값 상태와 버튼 이벤트 처리

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/ExpenseScreen.kt`

```

var transactionType by remember { mutableStateOf(TransactionType.EXPENSE) }
var amountText by remember { mutableStateOf("") }
var category by remember { mutableStateOf("식비") }
var emotion by remember { mutableStateOf("기쁨") }
var memo by remember { mutableStateOf("") }
var editingExpenseId by remember { mutableStateOf<Int?>(null) }

ExpenseForm(
    amountText = amountText,
    onAmountChange = { amountText = it },
    onSubmit = {
        val amount = amountText.toIntOrNull()
        if (amount != null && amount > 0) {
            if (editingExpenseId == null) {
                viewModel.addExpense(
                    amount, category, emotion, memo,
                    selectedDateMillis, transactionType
                )
            } else {
                viewModel.updateExpense(
                    editingExpenseId!!, amount, category, emotion, memo,
                    selectedDateMillis, transactionType
                )
            }
        }
        resetForm()
    }
)

```

`remember`와 `mutableStateOf`는 사용자가 입력한 값을 화면 상태로 보관한다. 입력창의 값이 바뀌면 `onAmountChange` 같은 이벤트가 실행되어 상태가 변경된다.

기록 추가 버튼을 누르면 `onSubmit`이 호출된다. 금액이 올바르게 입력되었는지 확인한 뒤, `editingExpenseId`가 없으면 새 기록을 추가하고 값이 있으면 기존 기록을 수정한다.

5.5 LazyColumn을 이용한 기록 목록

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/ExpenseScreen.kt`

```

val expenses by viewModel.expenses.collectAsState()
LazyColumn {

```

```

items(filteredExpenses, key = { it.id }) { expense ->
    ExpenseItem(
        expense = expense,
        onEdit = {
            amountText = expense.amount.toString()
            category = expense.category
            emotion = expense.emotion
            memo = expense.memo
            editingExpenseId = expense.id
        },
        onDelete = {
            pendingDeleteExpense = expense
        }
    )
}
}

```

`collectAsState()`는 ViewModel의 기록 목록을 화면에서 관찰한다. 기록이 추가되거나 삭제되면 목록이 바뀌고 화면도 다시 표시된다.

`LazyColumn`은 화면에 필요한 항목만 그리기 때문에 기록이 많아져도 목록을 스크롤하여 볼 수 있다. 각 기록은 `ExpenseItem`에 전달되어 수입·지출 유형, 금액, 날짜, 카테고리, 감정, 메모를 카드로 표시한다.

5.6 검색 조건을 이용한 기록 필터링

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/ExpenseScreen.kt`

```

val filteredExpenses = expenses.filter { expense ->
    val matchesQuery = appliedSearchText.isBlank() ||
        expense.type.label.contains(appliedSearchText, ignoreCase = true) ||
        expense.category.contains(appliedSearchText, ignoreCase = true) ||
        expense.emotion.contains(appliedSearchText, ignoreCase = true) ||
        expense.memo.contains(appliedSearchText, ignoreCase = true)

    val matchesType =
        typeFilter == "전체" || expense.type.label == typeFilter
    val matchesCategory =
        categoryFilter == "전체" || expense.category == categoryFilter
    val matchesEmotion =
        emotionFilter == "전체" || expense.emotion == emotionFilter
    val matchesAmount =
        (minAmountFilter == null || expense.amount >= minAmountFilter) &&
        (maxAmountFilter == null || expense.amount <= maxAmountFilter)

    matchesQuery && matchesType && matchesCategory &&
        matchesEmotion && matchesAmount
}

```

검색 기능은 Kotlin의 `filter`와 조건식을 사용한다. 검색어가 비어 있거나 기록의 유형, 카테고리, 감정, 메모에 검색어가 포함되면 검색어 조건을 만족한다. 유형, 카테고리, 감정, 최소·최대 금액 조건도 각각 계산한 뒤 모든 조건이 참인 기록만 결과 목록에 남긴다.

5.7 하단 탭을 이용한 화면 전환

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/EmotionBudgetAppRoot.kt`

```

private enum class AppDestination(val label: String) {
    Ledger("기록"),
    Report("통계"),
    Emotion("감정")
}

var currentDestination by remember {
    mutableStateOf(AppDestination.Ledger)
}

Scaffold(
    bottomBar = {
        NavigationBar {
            AppDestination.values().forEach { destination ->
                NavigationBarItem(
                    selected = currentDestination == destination,
                    onClick = { currentDestination = destination },
                    icon = { Text(destination.label) }
                )
            }
        }
    }
)

```

```

) { innerPadding ->
    when (currentDestination) {
        AppDestination.Ledger -> ExpenseScreen(viewModel)
        AppDestination.Report -> LedgerReportScreen(expenses, onBack = {})
        AppDestination.Emotion -> EmotionAnalysisScreen(expenses, onBack = {})
    }
}

```

하단 네비게이션 항목을 누르면 `onClick`에서 `currentDestination` 값이 바뀐다. `when`문은 현재 선택된 값에 따라 기록, 통계, 감정 화면 중 하나를 표시한다. 세 화면을 한곳에 길게 이어 붙이지 않고 탭으로 나누어 필요한 화면으로 바로 이동하게 했다.

5.8 월별 통계와 감정별 통계 계산

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/LedgerReportScreen.kt`,
`EmotionAnalysisScreen.kt`

```

val monthRecords = expenses.filter {
    it.dateMillis >= monthStart && it.dateMillis < monthEnd
}

val monthIncomeTotal = monthRecords
    .filter { it.type == TransactionType.INCOME }
    .sumOf { it.amount }

val monthExpenseTotal = monthRecords
    .filter { it.type == TransactionType.EXPENSE }
    .sumOf { it.amount }

val current = monthExpenses.filter { it.emotion == emotion }
val previous = previousMonthExpenses.filter { it.emotion == emotion }
val currentTotal = current.sumOf { it.amount }

EmotionStat(
    emotion = emotion,
    totalAmount = currentTotal,
    count = current.size,
    averageAmount = if (current.isEmpty()) 0 else currentTotal / current.size,
    previousCount = previous.size,
    countChange = current.size - previous.size
)

```

월별 통계는 선택한 달의 시작 시각과 다음 달 시작 시각 사이에 있는 기록만 `filter`로 가져온다. 이후 수입과 지출을 나누어 `sumOf`로 합계를 계산한다.

감정별 통계는 같은 감정의 기록을 모아 총금액, 횟수, 평균 금액을 계산한다. 이전 달의 기록 수도 함께 계산하여 감정별 소비 횟수가 증가했는지 확인할 수 있다.

5.9 날짜별 기록 묶기

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/LedgerReportScreen.kt`

```

private fun buildDailyGroups(records: List<Expense>): List<DailyRecordGroup> {
    return records
        .groupBy { startOfDay(it.dateMillis) }
        .map { (dayMillis, dayRecords) ->
            DailyRecordGroup(
                dayMillis = dayMillis,
                records = dayRecords.sortedByDescending { it.id },
                incomeTotal = dayRecords.sumByType(TransactionType.INCOME),
                expenseTotal = dayRecords.sumByType(TransactionType.EXPENSE)
            )
        }
        .sortedByDescending { it.dayMillis }
}

private fun List<Expense>.sumByType(type: TransactionType): Int {
    return filter { it.type == type }.sumOf { it.amount }
}

```

날짜에는 시와 분 정보도 포함되어 있으므로 그대로 비교하면 같은 날의 기록도 서로 다른 값으로 처리된다.

`startOfDay()`는 시간을 0시로 맞춰 같은 날짜가 같은 값이 되도록 만든다. 이후 `groupBy`로 날짜별 목록을 만들고, 각 그룹 안에서 수입과 지출 합계를 따로 계산한다.

완성된 그룹은 최신 날짜가 먼저 보이도록 정렬한다. 통계 화면의 일별 목록은 이 결과를 사용하므로, 한 날짜에 기록이 여러 개 있어도 날짜 제목과 하루 합계는 한 번만 표시된다.

5.10 감정 소비 진단 계산

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/EmotionAnalysisScreen.kt`

```
val monthTotal = monthExpenses.sumOf { it.amount }.takeIf { it > 0 } ?: 1
val stressCount = stressStat?.count ?: 0
val stressAmount = stressStat?.totalAmount ?: 0
val stressShare = stressAmount.toFloat() / monthTotal.toFloat()
val stressChange = stressStat?.countChange ?: 0

val baseScore = 20
val stressCountScore = (stressCount * 12).coerceAtMost(30)
val stressIncreaseScore = if (stressChange > 0) 20 else 0
val stressShareScore = if (stressShare >= 0.3f) 25 else 0
val primaryStressScore = if (primaryEmotion == "스트레스") 15 else 0
val riskScore = (
    baseScore +
    stressCountScore +
    stressIncreaseScore +
    stressShareScore +
    primaryStressScore
).coerceIn(0, 100)

val riskLabel = when {
    riskScore >= 70 -> "높음"
    riskScore >= 40 -> "보통"
    else -> "낮음"
}
```

감정 소비 진단은 이번 달 지출 중 스트레스 소비가 얼마나 반복되었는지를 점수로 바꾸는 부분이다. 스트레스 소비 횟수, 지난달보다 증가했는지 여부, 전체 지출에서 차지하는 비율, 가장 많은 지출 감정이 스트레스인지 여부를 각각 계산한 뒤 더한다.

점수는 `coerceIn(0, 100)`으로 0점에서 100점 사이를 유지한다. 계산된 점수에 따라 낮음, 보통, 높음으로 구분하고, 화면에서는 단계에 맞는 설명과 추천 행동을 보여준다. 따라서 사용자가 입력한 감정은 단순한 메모에 그치지 않고 실제 소비 분석 결과로 이어진다.